



Manual de Implementação - Fases 2-20



Objetivo

Transformar o design das 20 fases em código funcional, replicando o padrão da **Fase 1** e adicionando componentes UI específicos para cada mecânica.



Estrutura de Trabalho

Padrão Genérico (Fase N)

Para cada fase, você precisará de:

1. **Data File:** `frontend/src/data/faseN.js`
 - Narrativa + escolhas (se dilema)
 - OU puzzle/minigame data (se mecânica especial)
 2. **Component:** `frontend/src/components/FaseN.jsx` (se precisa UI especial)
 - Dilemas: Reutilizar `VisualNovelEngine.jsx`
 - Puzzles: Novo component
 - Minigames: Novo component
 3. **CSS:** `frontend/src/components/FaseN.css` (se novo component)
 - Tema cyberpunk consistente
 4. **Database:** Já está em `schema.sql` (20 fases pré-seeded)
-



Passo-a-Passo por Unidade

UNIDADE I: Bolha e Desinformação (Fase 1 pronta)

Fase 1: Prisioneiro da Bolha

-  Status: COMPLETO
- Mecânica: Dilema (3 opções)
- Component: `VisualNovelEngine.jsx` (genérico)

- Arquivo: `frontend/src/data/fase1.js`

Fase 2: Verdadeiro ou Falso?

- 📄 Status: DESIGN (roadmap20fases.js)
- Mecânica: Puzzle (reconhecer padrões)
- Component: `NOVO` → `PuzzleRecognition.jsx`
- Objetivo: Reconhecer manchetes falsas em 10 manchetes (5 fake, 5 real)

Implementação:

1. Criar arquivo data: `frontend/src/data/fase2.js`

```
export default {
  id_fase: 2,
  codigo: 'BOLHA_002',
  titulo: 'Verdadeiro ou Falso?',
  descricao: 'Identifique as 5 manchetes falsas entre 10',

  manchetes: [
    {
      id: 1,
      titulo: 'Estudo comprova benefício de eletrólitos para cérebro',
      falsa: true, // Fake news!
      dicas_visuais: ['fonte desconhecida', 'sem data', 'claim muito forte'],
      feedback: 'A falta de fonte confiável é um red flag. Sempre procure o .edu ou .gov'
    },
    {
      id: 2,
      titulo: 'OMS alerta sobre riscos de privacidade em aplicativos de saúde',
      falsa: false, // Verdadeira
      dicas_visuais: ['fonte confiável (OMS)', 'data recente', 'claim moderado'],
      feedback: 'Correto! Fontes oficiais + tópico relevante = confiável'
    },
    // ... mais 8 manchetes
  ],

  meta: 5 // Acertar pelo menos 5
}
```

2. Criar component: `frontend/src/components/PuzzleRecognition.jsx`

```

import { useState, useEffect } from 'react';
import { useGameProgress } from '../hooks/useGameProgress';
import fase2Data from '../data/fase2.js';
import './PuzzleRecognition.css';

export default function PuzzleRecognition({ idAluno, onFaseCompleta }) {
  const { registrarDecisao } = useGameProgress();
  const [manchetes, setManchetes] = useState(fase2Data.manchetes);
  const [selecionadas, setSelecionadas] = useState(new Set());
  const [feedback, setFeedback] = useState(null);
  const [acertos, setAcertos] = useState(0);

  const toggle = (id) => {
    const novo = new Set(selecionadas);
    if (novo.has(id)) novo.delete(id);
    else novo.add(id);
    setSelecionadas(novo);
  };

  const verificar = async () => {
    let count = 0;
    for (const id of selecionadas) {
      const m = manchetes.find(x => x.id === id);
      if (m.falsa) count++;
    }

    const classificacao = count >= fase2Data.meta ? 'correta' : 'catastrofica';
    const pontos = count >= fase2Data.meta ? 20 : 5;

    // Salvar decisão
    await registrarDecisao({
      idFase: 2,
      escolha: `puzzle_${count}_corretos`,
      categoria: 'logica',
      classificacao,
      pontoGanhos: pontos
    });

    setFeedback(classificacao);
    setAcertos(count);
  };

  return (
    <div className="puzzle-container">
      <h2>Identifique as 5 Manchetes Falsas</h2>
      <div className="manchetes-grid">
        {manchetes.map(m => (

```

```

    <div
      key={m.id}
      className={`manchete ${selecionadas.has(m.id) ? 'selecionada' : ''}`}
      onClick={() => toggle(m.id)}
    >
      <p>{m.titulo}</p>
      <span className="checkbox" />
    </div>
  )}}
</div>

<button onClick={verificar} className="botao-verificar">
  Verificar Resposta
</button>

{feedback && (
  <div className={`feedback ${feedback}`}>
    <p>Você acertou {acertos} de 5!</p>
    {feedback === 'correta' && <p>Excelente! Você identifica fake news!</p>}
    {feedback === 'catastrofica' && <p>Estude mais. Tenha atenção a fontes e datas.</p>}
    <button onClick={() => onFaseCompleta(2)}>Próxima Fase</button>
  </div>
)}
</div>
);
}

```

3. Criar CSS: frontend/src/components/PuzzleRecognition.css

```
.puzzle-container {
  max-width: 900px;
  margin: 40px auto;
  padding: 24px;
  background: linear-gradient(135deg, rgba(5, 1, 15, 0.95), rgba(26, 0, 51, 0.95));
  border: 1px solid #ff00aa;
  border-radius: 12px;
  color: #d6f5ff;
  font-family: 'Courier New', monospace;
}

.manchetes-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 12px;
  margin: 24px 0;
}

.manchete {
  background: rgba(0, 255, 200, 0.08);
  border: 2px solid rgba(0, 255, 200, 0.2);
  padding: 16px;
  border-radius: 6px;
  cursor: pointer;
  transition: all 0.3s;
}

.manchete:hover {
  border-color: #00ffc8;
  box-shadow: 0 0 20px rgba(0, 255, 200, 0.3);
}

.manchete.selecionada {
  background: rgba(255, 0, 170, 0.2);
  border-color: #ff00aa;
  box-shadow: 0 0 20px rgba(255, 0, 170, 0.3);
}

.manchete p {
  margin: 0;
  line-height: 1.5;
  font-size: 0.9rem;
}

.botao-verificar {
  width: 100%;
  padding: 12px;
```

```
background: linear-gradient(90deg, #00ffc8, #ff00aa);
border: none;
color: #050115;
font-weight: bold;
border-radius: 6px;
cursor: pointer;
font-size: 1rem;
}

.feedback {
margin-top: 24px;
padding: 16px;
border-radius: 6px;
}

.feedback.correta {
background: rgba(0, 255, 200, 0.1);
border-left: 4px solid #00ffc8;
}

.feedback.catastrofica {
background: rgba(255, 85, 102, 0.1);
border-left: 4px solid #ff5566;
}
```

Resumo: Como Replicar Padrão

Para DILEMA (Fases 1, 4, 5, 8, 9, 12, 13, 16, 17, 20):

1. Criar `faseN.js` com dialogo + escolhas
2. Usar `VisualNovelEngine.jsx` (genérico, sem novo component)
3. Hook: `useGameProgress.registrarDecisao()`

Para PUZZLE (Fases 2, 6, 10, 14, 18):

1. Criar `faseN.js` com dados do puzzle
2. Criar `PuzzleN.jsx` (novo component)
3. Criar `PuzzleN.css` (tema cyberpunk)
4. Hook: `useGameProgress.registrarDecisao()`

Para MINIGAME (Fases 3, 7, 11, 15, 19):

1. Criar `faseN.js` com dados da mecânica
2. Criar `MinigameN.jsx` (novo component)

3. Criar `MinigameN.css` (tema cyberpunk)
 4. Hook: `useGameProgress.registrarDecisao()`
-

Checklist de Implementação

Unidade I (Fases 1-4)

- ☒ Fase 1: Dilema - VisualNovelEngine
- ☐ Fase 2: Puzzle - PuzzleRecognition
- ☐ Fase 3: Minigame - AlgoritmoDecomposição
- ☐ Fase 4: Dilema x5 - VisualNovelEngine (5 scenes)
- ☐ Chamar `calcular_perfil.php` após Fase 4

Unidade II (Fases 5-8)

- ☐ Fase 5: Dilema - VisualNovelEngine
- ☐ Fase 6: Puzzle - AnaliseDataset
- ☐ Fase 7: Minigame - EquidadeVsPrecisão
- ☐ Fase 8: Dilema x4 - VisualNovelEngine
- ☐ Chamar `calcular_perfil.php` após Fase 8

Unidade III (Fases 9-12)

- ☐ Fase 9: Dilema - VisualNovelEngine
- ☐ Fase 10: Puzzle - DetectarArtefatos
- ☐ Fase 11: Minigame - ForenseDigital
- ☐ Fase 12: Dilema x3 - VisualNovelEngine
- ☐ Chamar `calcular_perfil.php` após Fase 12

Unidade IV (Fases 13-16)

- ☐ Fase 13: Dilema - VisualNovelEngine
- ☐ Fase 14: Puzzle - QuebrarSenhas
- ☐ Fase 15: Minigame - Criptografia
- ☐ Fase 16: Dilema x3 - VisualNovelEngine
- ☐ Chamar `calcular_perfil.php` após Fase 16

Unidade V (Fases 17-20)

- ☐ Fase 17: Dilema - VisualNovelEngine
- ☐ Fase 18: Puzzle - TeiaDePoderCorporativa
- ☐ Fase 19: Minigame - SoftwareOutros
- ☐ Fase 20: Dilema x5 (FINAL) - VisualNovelEngine
- ☐ Chamar `calcular_perfil.php` após Fase 20 ← FINAL PROFILE



Integração: Quando Chamar `calcular_perfil.php`

Após terminar Fases **4, 8, 12, 16, 20** (fim de cada unidade):

```
// Em VisualNovelEngine.jsx ou novo component
useEffect(() => {
  if ([4, 8, 12, 16, 20].includes(faseAtual)) {
    // Chamar cálculo de perfil
    fetch(`${API_BASE}/calcular_perfil.php`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        id_aluno: idAluno,
        fase_alcancada: faseAtual
      })
    })
    .then(res => res.json())
    .then(data => {
      if (data.sucesso) {
        // Mostrar <PerfilAluno /> component
        setMostrarPerfil(true);
        setPerfil(data);
      }
    });
  }
}, [faseAtual]);

if (mostrarPerfil) {
  return <PerfilAluno idAluno={idAluno} faseAlcancada={faseAtual} />;
}
```


Template: Criar Nova Fase (Dilema)

File: frontend/src/data/faseN.js

```
export default {
  id_fase: 5,
  codigo: 'VIES_001',
  titulo: 'Reconhecimento Facial Enviesado',

  dialogo: [
    { personagem: 'SISTEMA', texto: 'Você é convidado para aprovar um sistema de reconhecimento' },
    { personagem: 'VOCÊ', texto: 'Mas e se ele tiver viés?' },
    { personagem: 'CEO', texto: 'Problema de quem usará, não nosso.' },
  ],

  escolhas: [
    {
      id_decisao: 'rejeita_sistema',
      texto: 'Rejeitar (denunciar o viés)',
      classificacao: 'correta',
      pontos_ganhos: { etica: 20, logica: 5, seguranca: 10 },
      feedback: 'Excelente! Você defendeu direitos.'
    },
    {
      id_decisao: 'negocia',
      texto: 'Negociar (pedir ajustes)',
      classificacao: 'neutra',
      pontos_ganhos: { etica: 10, logica: 15, seguranca: 5 },
      feedback: 'Pragmático. Mas melhorias bastam?'
    },
    {
      id_decisao: 'aceita',
      texto: 'Aceitar (cumprir CEO)',
      classificacao: 'catastrofica',
      pontos_ganhos: { etica: 0, logica: 5, seguranca: 0 },
      feedback: 'Prejudicial a minorias. Consequências reais.'
    }
  ]
}
```

Use em VisualNovelEngine:

```
<VisualNovelEngine
  idAluno={1}
  faseData={faseN} // Passar data prop
  onFaseCompleta={handler}
/>
```

Sequência Recomendada de Implementação

Semana 1: Fases 2-4 (Unidade I)

- Domingo: Code Fase 2 + 3
- Segunda: Testar Fase 2 + 3
- Terça: Refinar + Fase 4
- Quarta: Integrar perfil após Fase 4

Semana 2: Fases 5-8 (Unidade II)

- Replicar padrão
- Testar

Semana 3: Fases 9-12 (Unidade III)

- Replicar padrão
- Testar

Semana 4: Fases 13-16 (Unidade IV)

- Replicar padrão
- Testar

Semana 5: Fases 17-20 (Unidade V)

- Replicar padrão
- Testar

Semana 6: Deploy + Testes

- Hostinger staging
 - Testes de aceitação
 - Ajustes finais
-

Exemplos Específicos por Mecânica

Puzzle Típico

Entrada: Conjunto de 10-15 items

Tarefa: Identificar X items corretos

Output: `registrarDecisao({ acertos: N, categoria: 'logica', ... })`

Exemplos:

- Fase 2: 10 manchetes → 5 fake (verdadeiro/falso)
- Fase 6: 10 datasets → 3 enviesados (categorias)
- Fase 10: 10 imagens → 5 com artefatos (pontos)
- Fase 14: 8 senhas → 3 fracas (senha fraca?)
- Fase 18: 12 conexões → 7 de influência (conexão verdadeira?)

Minigame Típico

Entrada: Sequência ou desafio de N etapas

Tarefa: Completar etapas em ordem

Output: `registrarDecisao({ etapas_completas: N, categoria: 'logica'/'seguranca', ... })`

Exemplos:

- Fase 3: Quebrar algoritmo em 5 etapas (decomposição)
- Fase 7: Ajustar 3 parâmetros (equidade vs precisão)
- Fase 11: Analisar imagem em 4 técnicas (análise forense)
- Fase 15: Criptografar mensagem com 3 métodos (Caesar→RSA)
- Fase 19: Construir app escolhendo 5 libs (open vs proprietário)

Validação (Teste Manual)

Para cada nova fase:

1. **Fase Carrega:** Login → Ver Fase N → Narrativa completa?
 2. **Escolhas Funcionam:** Clicar botão → Registra decisão?
 3. **XP Atualiza:** HUD mostra novo XP?
 4. **DB Persiste:** `SELECT log_decisoes WHERE id_fase=N;` → Dados aí?
 5. **Próxima Fase:** Botão "Próxima" leva ao menu de fases?
-

Quando terminar Fase 20:

- ☒ 20 fases implementadas
- ☒ Todas as decisões no BD
- ☒ Perfil calculado 5x (após 4, 8, 12, 16, 20)
- ☒ Aluno vê seu tipo de Cidadão Digital
- ☒ Professor tem relatório personalizado

Parabéns! Você criou um jogo pedagógico completo de 20 fases!

Versão: 1.0

Data: 02/09/2026

Pronto para começar?